Basic (typed function + custom error) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Basic` section — the same file also holds the Failure handling exercise, under a `# Failure handling` section. Run it with `cd practice && python custom_errors_practice.py`.

Used later by: the Real-world wiring exercise and the Build Task. Neither imports this file — they re-write the same one-line custom-error pattern as `MissingConfigError` in `practice/config_logging_wiring/exceptions.py` and `practice/build_task/exceptions.py`.

Story — `custom_errors_practice.py` (Basic section): this is the smallest possible version of a pattern used everywhere later — a named exception class instead of a generic one, raised the moment something's actually wrong. Written this way so the habit ("name the failure, raise it right at the source") is muscle memory before Doc01's Build Task needs it for real. **If not:** every later exercise's first encounter with a custom error would be inside a bigger, more distracting file (config loading, logging setup), making it harder to see the pattern in isolation.

Solution

```
# practice/custom_errors_practice.py
class InvalidAgeError(Exception):
    # why: a specific error type lets calling code catch "bad age" on its
    # own, instead of catching every possible Exception blindly.
    pass

def set_age(age: int) -> None:
    # when: call this any time an age value comes from outside the program
    # (user input, a form, a file) — not for values you already trust.
    if age < 0:
        # how: raising here, at the point the bad value is found, is what
        # makes the traceback point at the real cause, not somewhere later.
        raise InvalidAgeError(f"age cannot be negative: {age}")

set_age(25)  # runs fine, no output — 25 is a valid age

try:
    set_age(-5)
```

```
except InvalidAgeError as e:
    # how: catching the specific error type, not bare Exception, means
    # only this exact problem gets handled here — anything else still crashes
    # loudly.
    print(f"Caught it: {e}")
```

Expected output:

```
Caught it: age cannot be negative: -5
```